

Luxe Stay 酒店预订与信用管理平台

系统详细设计说明书

酒店核心业务系统底层详细架构、功能模块及核心算法设计

一、引言

1.1 系统简介

Luxe Stay 酒店在线预订与信用管理平台是一套基于 Spring Boot + Thymeleaf + SQL Server 的轻量级单体酒店信息化预订系统。系统涵盖了完整的用户权限认证隔离体系，并融合了极具课设特色的信用违约检测机制（即对连续取消订单用户实施自动期限禁订），确保酒店的房型库存在并发销售中维持高效周转。

1.2 编写目的

本说明书旨在对 Luxe Stay 平台的总体设计理念、技术选型、数据库物理设计、核心功能模块（用户登录/授权拦截/每日库存级联校验/事务下单及信用封禁）的内部代码逻辑及核心控制算法进行详细定义，为开发人员的具体编码与接口集成测试提供最终准则。

1.3 开发环境与运行环境配置

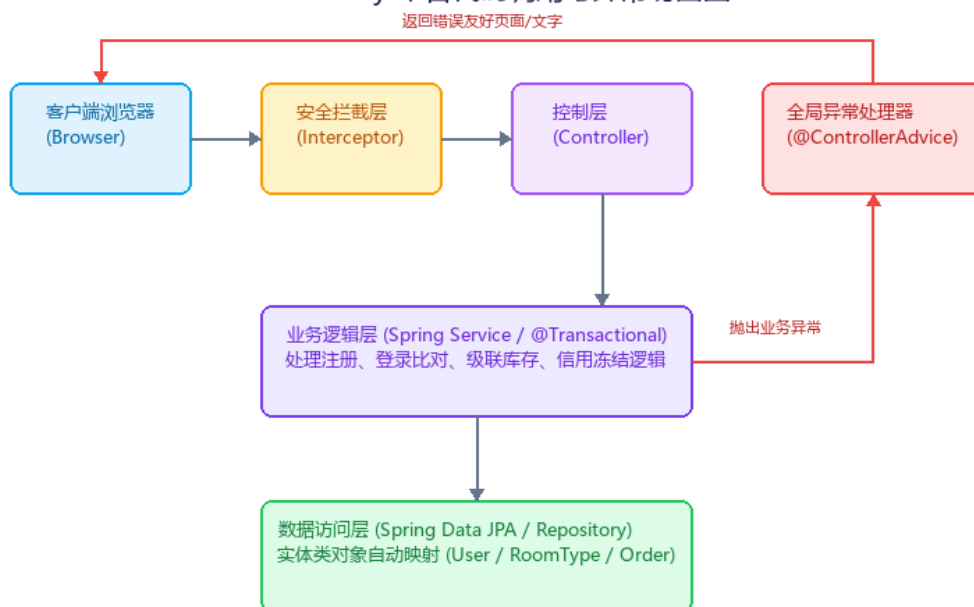
- 开发环境：**JDK 17、Maven 3.8+、IntelliJ IDEA 2024.1 (Ultimate Edition)、Git 版本协作控制工具。
- 运行环境：**服务器部署于 Windows Server 2019/2022 操作系统中，内部采用内置 Tomcat 9 Web 容器方式搭载编译生成的 Jar 运行包，后台连接部署于 localhost:1433 端口的 Microsoft SQL Server 2025 Express 数据库。

二、系统总体设计

2.1 系统代码调用流图

系统采用了分层的 MVC 开发架构，请求从客户端发出，在后端通过控制器、服务、实体和持久化组件，结合全局异常处理器以非侵入方式处理所有抛出的业务异常。架构流程图如下：

Luxe Stay 平台代码调用与异常切面图



2.2 功能模块划分及调用关系

系统在代码包结构中明确划分为三大职责明确的核心包结构模块：

- 用户与权限管理包 (com.hotel.system.config/controller/entity/repository/service)：管理用户信息、密码加密、Session 登录状态、越权请求路由拦截、信用违约封禁状态动态自动解除。
- 房型与库存管理包 (com.hotel.system.entity/repository/service)：管理房型的基本 JPA CRUD、每日具体日期的库存数量调整、按时间跨度校验多天连续可用库存。
- 预订与订单控制包 (com.hotel.system.controller/service)：处理多晚预订下单、并发级联扣减每日库存（加写锁防止超卖）、单价快照存储、已定订单在入住日前合理退订及库存原数归还。

三、核心功能模块详细设计

3.1 用户注册登录与安全拦截模块

- 模块名称：用户与权限安全管理模块
- 模块功能：提供旅客注册（调用 BCrypt Hashing 进行单向密码加密）；安全登录与 Session 认证；基于 HandlerInterceptor 实现未登录请求全局拦截与普通会员对管理员路径（`/admin/\*\*`）越权访问拦截重定向；个人中心展示信用度量与违约次数。
- 业务流程与逻辑描述：用户输入用户名密码后，系统经过拦截器，若无 Session 则重定向登录；在 Controller 接收后调用 Service 比对密文，成功后绑定 Session，否则向前端模型抛出错误。若普通用户直接尝试访问后台 `/admin/\*\*` 路径，则重定向至 `403` 页面。

- **业务数据库操作：**对 `users` 表的行级 INSERT（注册）、查询（登录校验）与 UPDATE（更新 cancel_count 与 banned_until 限制时间）。
- **异常处理机制：**当用户注册输入了已被占用的重复用户名时，Service 抛出 `IllegalArgumentException` 自定义业务异常，全局异常切面捕获该异常并将其解析返回到注册页前端进行字段级报错。

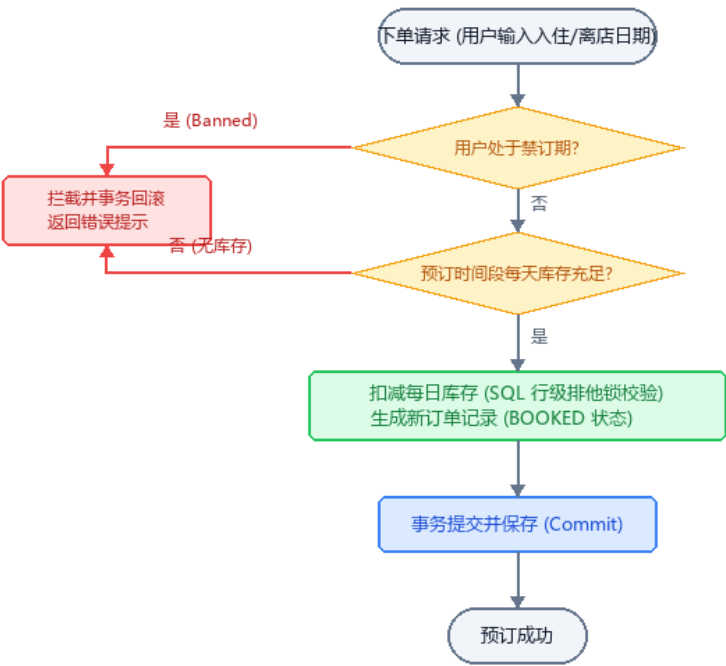
3.2 房型管理与每日库存模块

- **模块名称：**房型与每日库存调度模块
- **模块功能：**提供管理员后台对房型基本信息的修改、房型可用状态标记（status）；提供管理员对每一天（如 2026-07-13）特定房型的库存做配置增改；用户在前台输入入住/离店日期后，系统逐日计算库存并进行可用性过滤。
- **业务流程与逻辑描述：**系统根据输入的入住和退房日期（离店日不占库存），动态对数据库 `room\_inventory` 对应日期行的 `available\_quantity` 进行提取和核对。若该期间任意一天的可用数量为 0，则该房型在前台检索时自动被过滤隐藏。
- **业务数据库操作：**对 `room\_type` 物理表的 CRUD；对 `room\_inventory` 物理表按照 `room\_type\_id` 和具体 `inventory\_date` 进行查询和单日可用配额更新。
- **异常处理机制：**若用户查询的入住离店范围超过了系统后台所配置的未来 7 天的最大库存天数限制，系统抛出超限异常，拦截预订逻辑。

3.3 酒店订单创建与事务库存扣减模块

- **模块名称：**预订交易与订单事务控制模块
- **模块功能：**处理用户的多日连续房间预订请求；校验库存；在单个 Spring 事务（`@Transactional`）中级联扣减对应日期范围的各天库存并生成 BOOKED 订单；保存订单快照单价以隔绝后期价格调动影响；处理取消预订、原数增加还回库存，并累加信用取消次数限制。
- **业务流程流图：**预订流程包含严密的安全与数据校验。详细的事务下单和库存扣除业务流程图如下：

Luxe Stay 事务级预订下单与库存扣减流程图



• **异常处理与事务回滚：**下单时，系统检查如果其中任何一天出现剩余可用库存不足的情况，抛出自定义的 `OutOfStockException` 并回滚事务。由于使用了 `@Transactional` 注解，之前的库存扣减与订单生成动作将全部被自动撤销，保证了数据库状态的强一致性。

四、数据库设计 (Database Design)

4.1 物理数据表字段规格说明

系统数据库内共设计 4 张物理表，其核心设计细节如下表所示：

■ 用户表 users

字段名	数据类型	约束/Key	允许为空	字段含义说明
id	BIGINT	PK (Identity)	NO	自增主键
username	VARCHAR(50)	UNIQUE	NO	登录用户名
password	VARCHAR(100)	-	NO	BCrypt 加密密码哈希值
role	VARCHAR(20)	CHECK	NO	角色权限 (USER / ADMIN)
banned_until	DATETIME2	-	YES	冻结预订截止时间 (NULL 为

正常)

■ 房型表 room_type

字段名	数据类型	约束/Key	允许为空	字段含义说明
id	BIGINT	PK (Identity)	NO	自增主键
name	NVARCHAR(100)	-	NO	房型展示名称
price	DECIMAL(10,2)	-	NO	基准每晚价格
total_quantity	INT	-	NO	该房型总房间配额数
status	INT	-	NO	启用标记 (1:启用, 0:禁用)

■ 每日库存表 room_inventory

字段名	数据类型	约束/Key	允许为空	字段含义说明
id	BIGINT	PK (Identity)	NO	自增主键
room_type_id	BIGINT	FK -> room_type.id	NO	关联房型外键
inventory_date	DATE	UNIQUE(type_id, date)	NO	具体库存日期
available_quantity	INT	-	NO	当天剩余可用房间数

■ 预订订单表 booking_order

字段名	数据类型	约束/Key	允许为空	字段含义说明
id	BIGINT	PK (Identity)	NO	自增主键
order_no	VARCHAR(50)	UNIQUE	NO	系统唯一订单编号
user_id	BIGINT	FK -> users.id	NO	预订用户外键
unit_price	DECIMAL(10,2)	-	NO	下单时的价格快照

status	VARCHAR(20)	CHECK	NO	订单状态 (BOOKED/CANCELLED/COMPLETED)
--------	-------------	-------	----	--------------------------------------

五、数据交互接口详细设计 (API Specification)

■ API 1: 用户身份登录校验

- 请求方法：POST
- 服务路由：/login
- 详细规约描述：

请求报文 (Request Parameters):

- username: String (必填，登录账号)
- password: String (必填，明文密码)

返回报文 (Response Details):

- 验证成功 (Http Status 302): 绑定 Session 状态，重定向至 /profile 页面。
- 验证失败 (Http Status 200): 页面不跳转，回传 error 错误提示文本。

■ API 2: 旅客跨天提交预订订单

- 请求方法：POST
- 服务路由：/booking/create
- 详细规约描述：

请求报文 (Request Parameters):

- roomId: Long (必填，房型实体 ID)
- checkIn: Date (格式 yyyy-MM-dd，入住日期)
- checkOut: Date (格式 yyyy-MM-dd，退房日期)
- quantity: Integer (必填，预订间数)

返回报文 (Response Details):

- 创建成功 (Http Status 200): 返回新建订单实体的 JSON 数据串。
- 失败拦截 (Http Status 400): 返回错误原因描述文本（如：用户在禁订期内，或部分日期房源库存不足）。

■ API 3: 会员取消订单申请

- 请求方法：POST
- 服务路由：/booking/cancel/{id}
- 详细规约描述：

请求报文 (Request Parameters):

- id: Long (路径参数, 需要取消的订单主键 ID)

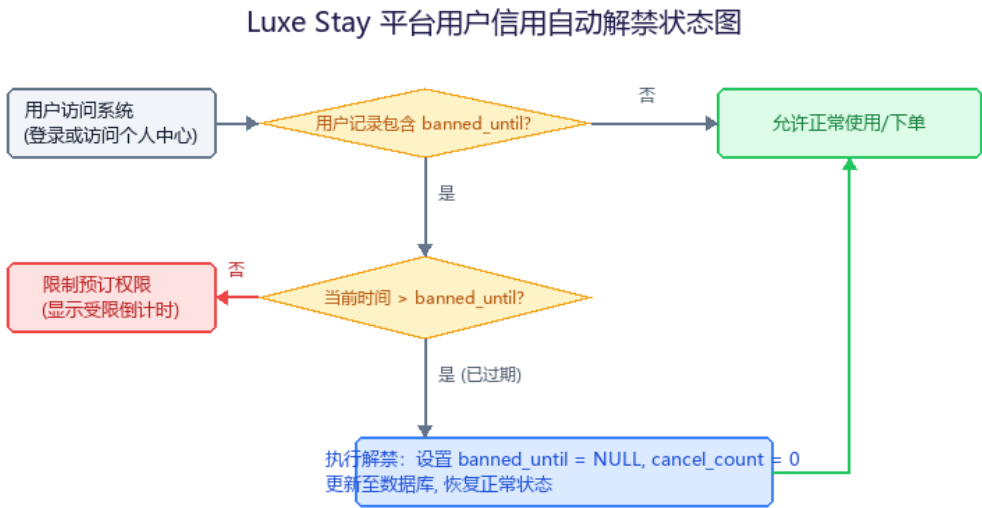
返回报文 (Response Details):

- 取消成功 (Http Status 302): 重定向回 /profile 会员中心。系统在底层将订单 status 设为 CANCELLED, 库存按日期原数返还, 且该用户的 cancel_count 计数器累加 1。

六、核心关键业务算法设计

6.1 信用状态检查与自动解禁机制

为最大程度减小系统后台在定时循环扫表时造成的资源开销, Luxe Stay 平台在信用封禁解除机制中采用了**“被动触发式解禁状态机”**。该状态检查触发时机为: 用户登录系统时, 或登录后每次刷新 /profile 会员中心时。被动式检测流程图如下:



其核心解禁验证的 Java 伪代码逻辑如下所示:

```
// 被动式信用解禁校验算法伪代码
public boolean checkAndUpdateBannedStatus(User user) {
    if (user.getBannedUntil() == null) {
        return false; // 当前未处于封禁状态
    }
    LocalDateTime now = LocalDateTime.now();
    if (now.isAfter(user.getBannedUntil())) {
        // 禁订期已过, 在事务中重置状态
        user.setBannedUntil(null);
        user.setCancelCount(0);
        userRepository.save(user);
    }
}
```

```
        return false; // 已被成功解禁，返回正常状态
    }
    return true; // 依然在封禁期内
}
```